

E-Commerce Data Analysis Report

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns
✓ [1] 19ms
```

```
import pandas as pd
```

```
orders = pd.read_csv(
    r"C:\Users\swati\OneDrive\Desktop\Meritshot Practice\Growth Skale\Project_4\Project 6\olist_orders_dataset.csv"
)

order_items = pd.read_csv(
    r"C:\Users\swati\OneDrive\Desktop\Meritshot Practice\Growth Skale\Project_4\Project 6\olist_order_items_dataset.csv"
)

products = pd.read_csv(
    r"C:\Users\swati\OneDrive\Desktop\Meritshot Practice\Growth Skale\Project_4\Project 6\olist_products_dataset.csv"
)

customers = pd.read_csv(
    r"C:\Users\swati\OneDrive\Desktop\Meritshot Practice\Growth Skale\Project_4\Project 6\olist_customers_dataset.csv"
)

print(orders.head())

print(orders.shape)

print(orders.columns)

✓ [4] 617ms
```

	order_id	customer_id \
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c

	order_status	order_purchase_timestamp	order_approved_at \
0	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15
1	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27
2	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23
3	delivered	2017-11-18 10:02:06	2017-11-18 10:45:59

####Checking the data types

9 10

```
print(orders.dtypes)
```

```
print(order_items.dtypes)
```

```
print(products.dtypes)
```

```
print(customers.dtypes)
```

✓ [5] 51ms

```
order_id          str
customer_id       str
order_status      str
order_purchase_timestamp  str
order_approved_at str
order_delivered_carrier_date str
order_delivered_customer_date str
order_estimated_delivery_date str
dtype: object
order_id          str
order_item_id     int64
product_id        str
```

####Checking the Null values

Check missing values for each dataset

```
print("Orders Missing Values:\n", orders.isnull().sum(), "\n")
```

```
print("Order Items Missing Values:\n", order_items.isnull().sum(), "\n")
```

```
print("Products Missing Values:\n", products.isnull().sum(), "\n")
```

```
print("Customers Missing Values:\n", customers.isnull().sum(), "\n")
```

```
orders.isnull().mean() * 100
```

✓ [6] 126ms

```
customer_state      0
dtype: int64

order_id          0.000000
customer_id       0.000000
order_status      0.000000
order_purchase_timestamp  0.000000
order_approved_at  0.160899
order_delivered_carrier_date  1.793023
order_delivered_customer_date  2.981668
order_estimated_delivery_date  0.000000
dtype: float64
```

```
####Find Duplicate
print(orders.duplicated().sum())

print(order_items.duplicated().sum())

print(products.duplicated().sum())

print(customers.duplicated().sum())
```

```
orders = orders.drop_duplicates()
```

```
✓ [7] 378ms
```

```
0
0
0
0
```

```
# Convert date columns in orders dataset
```

```
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])
orders['order_delivered_customer_date'] = pd.to_datetime(orders['order_delivered_customer_date'])
orders['order_estimated_delivery_date'] = pd.to_datetime(orders['order_estimated_delivery_date'])
```

```
✓ [8] 84ms
```

```
# Check earliest and latest order dates
```

```
print("Earliest order date:", orders['order_purchase_timestamp'].min())
print("Latest order date:", orders['order_purchase_timestamp'].max())
```

```
✓ [9] 46ms
```

```
Earliest order date: 2016-09-04 21:15:19
Latest order date: 2018-10-17 17:30:18
```

```
# Example: fill missing delivery dates with 'Not Delivered'
```

```
orders['order_delivered_customer_date'] = orders['order_delivered_customer_date'].fillna("Not Delivered")
```

```
✓ [10] 58ms
```

```
# Example: drop rows where product category is missing
```

```
products = products.dropna(subset=['product_category_name'])
```

```
✓ [11] 23ms
```

```
# Check product price
```

```
print("Negative prices:", (order_items['price'] < 0).sum())
print("Zero prices:", (order_items['price'] == 0).sum())
```

```
✓ [12] 36ms
```

```
Negative prices: 0
Zero prices: 0
```

```
# Check freight value
```

```
print("Negative freight:", (order_items['freight_value'] < 0).sum())
```

```
✓ [13] 34ms
```

```
# Check quantity
print("Zero or negative quantity:", (order_items['order_item_id'] <= 0).sum())
```

✓ [14] 36ms

Zero or negative quantity: 0

```
# Remove rows with invalid prices or freight
order_items = order_items[(order_items['price'] > 0) & (order_items['freight_value'] >= 0)]
```

✓ [15] 15ms

```
# Total rows
print("Total Orders:", len(orders))
print("Total Customers:", len(customers))
print("Total Products:", len(products))
```

✓ [16] 39ms

Total Orders: 99441
Total Customers: 99441
Total Products: 32341

```
# Unique counts
print("Unique Orders:", orders['order_id'].nunique())
print("Unique Customers:", customers['customer_id'].nunique())
print("Unique Products:", products['product_id'].nunique())

status_counts = orders['order_status'].value_counts()
status_percentage = orders['order_status'].value_counts(normalize=True) * 100

print("Order Status Counts:\n", status_counts)
print("\nOrder Status Percentage:\n", status_percentage)
```

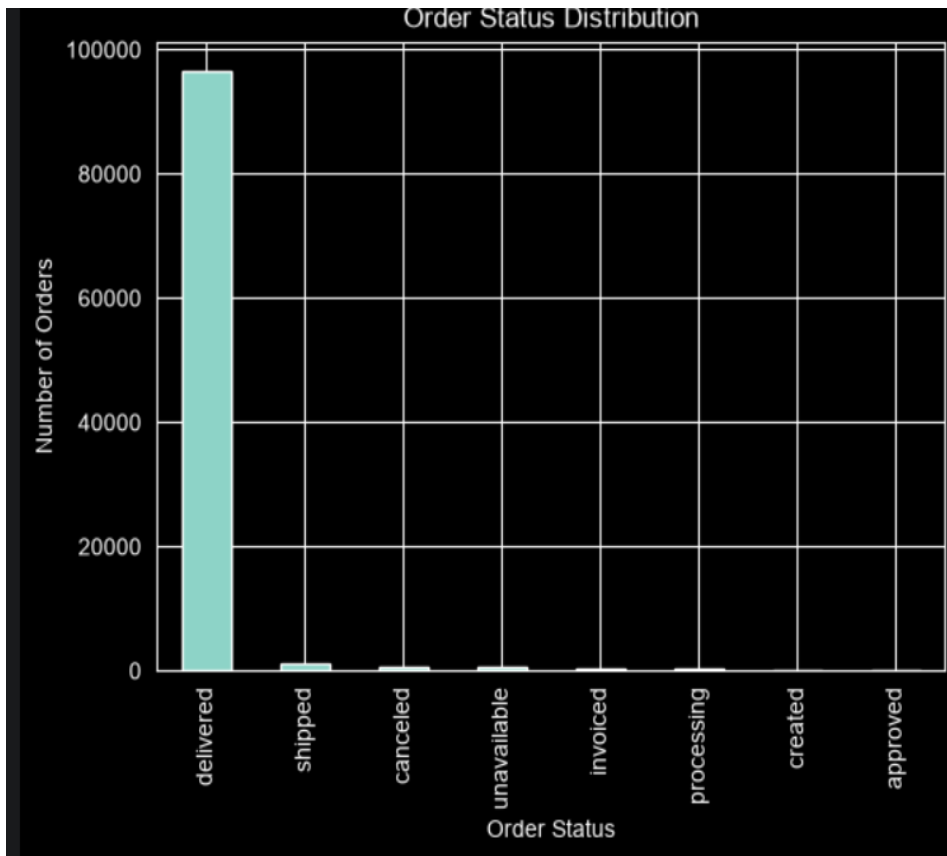
```
import matplotlib.pyplot as plt
```

```
status_counts.plot(kind='bar', title='Order Status Distribution')
plt.xlabel("Order Status")
plt.ylabel("Number of Orders")
plt.show()
```

✓ [17] 246ms

Unique Orders: 99441
Unique Customers: 99441
Unique Products: 32341
Order Status Counts:

order_status	
delivered	96478
shipped	1107
canceled	625
unavailable	609
invoiced	314
processing	301
created	5



```
# Total product sales
total_sales = order_items['price'].sum()
✓ [18] 18ms

# Total freight value
total_freight = order_items['freight_value'].sum()
✓ [19] 40ms

# Average product price
avg_price = order_items['price'].mean()
✓ [20] 42ms

# Average freight value
avg_freight = order_items['freight_value'].mean()

print("Total Product Sales:", total_sales)
print("Total Freight Value:", total_freight)
print("Average Product Price:", avg_price)
print("Average Freight Value:", avg_freight)
✓ [21] 34ms

Total Product Sales: 13591643.7
Total Freight Value: 2251909.54
Average Product Price: 120.65373901464713
Average Freight Value: 19.990319928983578
```

```
# Merge order_items with products
items_products = order_items.merge(products, on='product_id', how='left')
✓ [22] 38ms
```

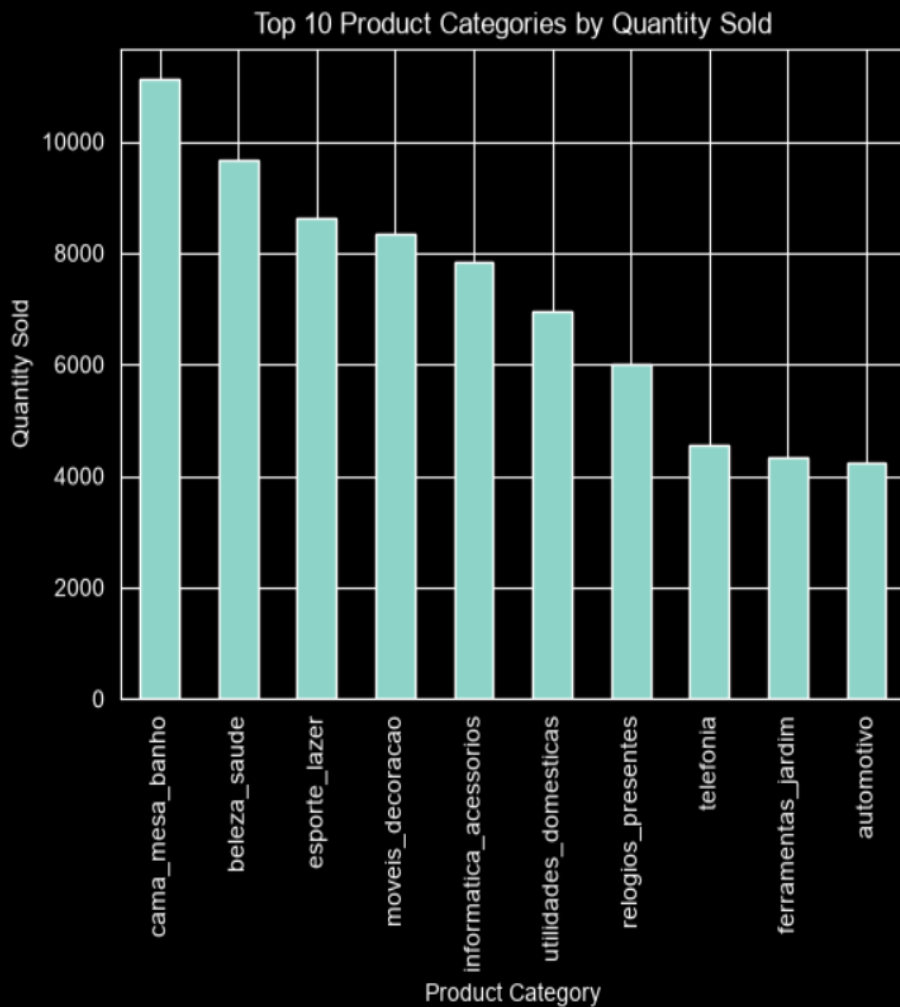
```
# Group by category and sum quantity
category_sales = items_products.groupby('product_category_name')['order_item_id'].count().sort_values(ascending=False)
✓ [23] 25ms
```

```
# Top 10 categories
top10_categories = category_sales.head(10)
print(top10_categories)
✓ [24] 33ms
```

product_category_name	
cama_mesa_banho	11115
beleza_saude	9670
esporte_lazer	8641
moveis_decoracao	8334
informatica_acessorios	7827
utilidades_domesticas	6964
relogios_presentes	5991
telefonia	4545
ferramentas_jardim	4347
automotivo	4235

Name: order_item_id dtype: int64

```
# Bar chart
top10_categories.plot(kind='bar', title='Top 10 Product Categories by Quantity Sold')
plt.xlabel("Product Category")
plt.ylabel("Quantity Sold")
plt.show()
✓ [25] 185ms
```

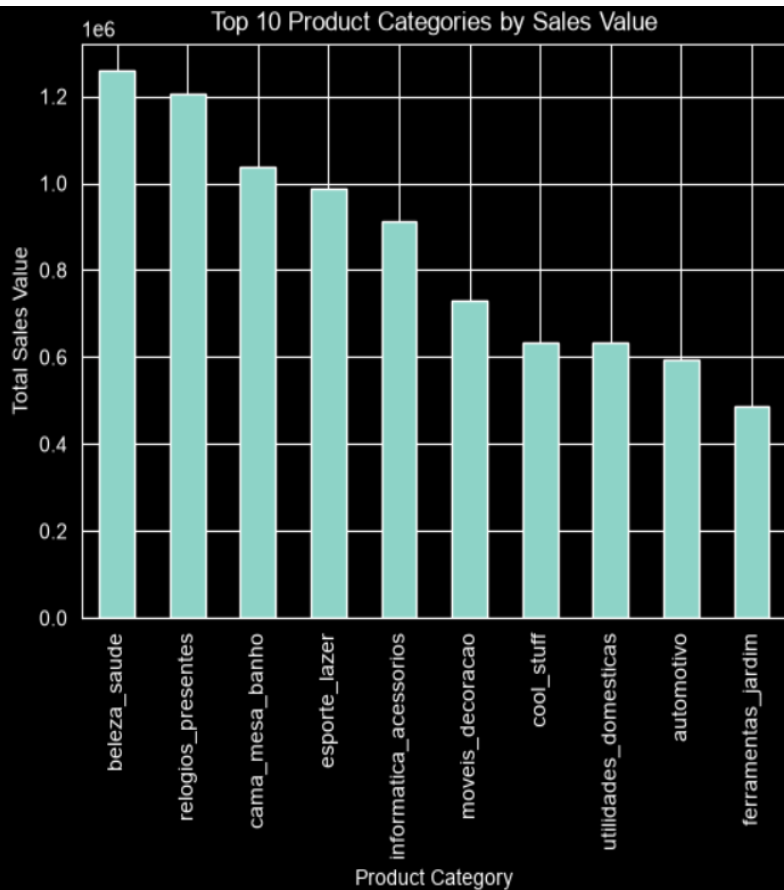


```
1 # Calculate sales per category
2 category_revenue = items_products.groupby('product_category_name')['price'].sum().sort_values(ascending=False)
✓ [26] 34ms
```

```
1 # Top 10 categories
2 top10_revenue = category_revenue.head(10)
3 print(top10_revenue)
✓ [27] 36ms
```

```
product_category_name
beleza_saude          1258681.34
relogios_presentes    1205005.68
cama_mesa_banho        1036988.68
esporte_lazer          988048.97
informatica_acessorios 911954.32
moveis_decoracao       729762.49
cool_stuff             635290.85
utilidades_domesticas 632248.66
automotivo             592720.11
ferramentas_jardim     485256.46
Name: price, dtype: float64
```

```
1 # Bar chart
2 top10_revenue.plot(kind='bar', title='Top 10 Product Categories by Sales Value')
3 plt.xlabel("Product Category")
4 plt.ylabel("Total Sales Value")
5 plt.show()
✓ [28] 215ms
```



```
# Quantity sold per product
product_quantity = items_products.groupby('product_id')['order_item_id'].count().sort_values(ascending=False).head(10)
```

✓ [29] 48ms

```
# Sales value per product
product_sales = items_products.groupby('product_id')['price'].sum().sort_values(ascending=False).head(10)
```

```
print("Top 10 Products by Quantity:\n", product_quantity)
print("\nTop 10 Products by Sales Value:\n", product_sales)
```

✓ [30] 51ms

Top 10 Products by Quantity:

product_id	quantity
aca2eb7d00ea1a7b8ebd4e68314663af	527
99a4788cb24856965c36a24e339b6058	488
422879e10f46682990de24d770e7f83d	484
389d119b48cf3043d311335e499d9c6b	392
368c6c730842d78016ad823897a372db	388
53759a2ecddad2bb87a079a1f1519f73	373
d1c427060a0f73f6b889a5c7c61f2ac4	343
53b36df67ebb7c41585e8d54d6772e08	323
154e7e31ebfa092203795c972e5804a6	281
3dd2a17168ac895c781a9191c1a95ad7	274

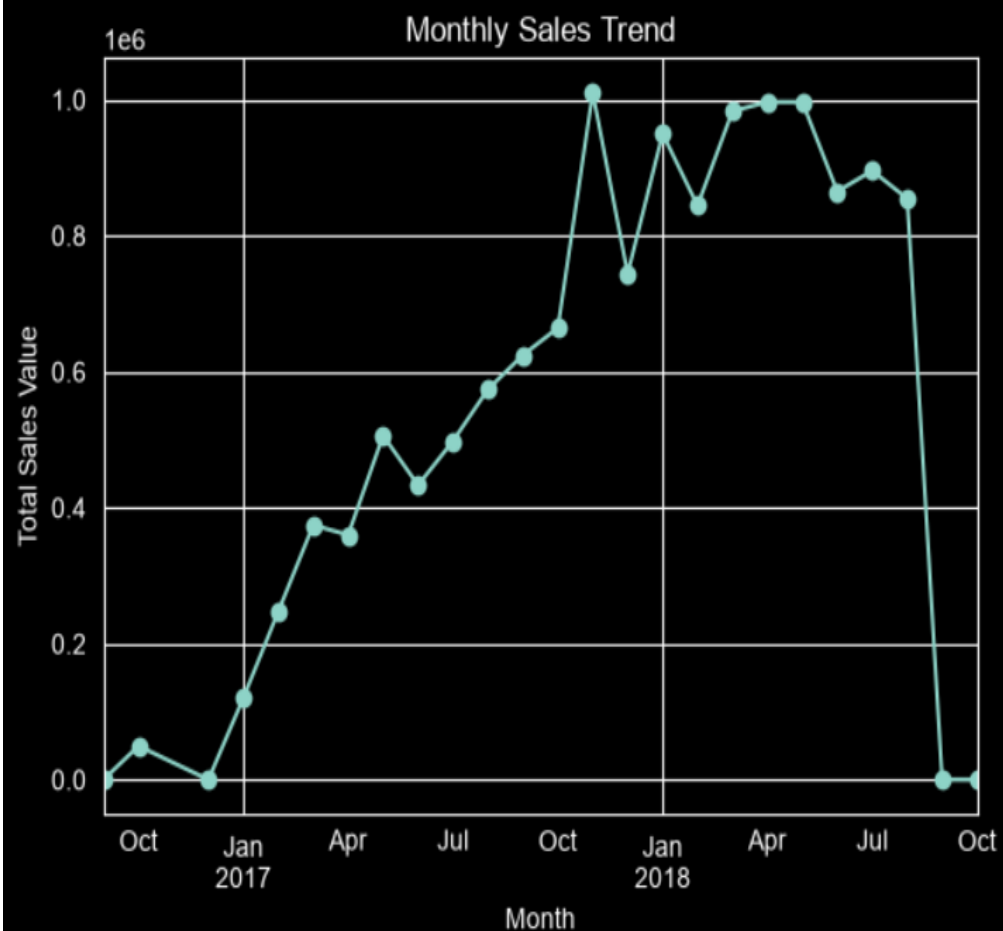
```
# Add month column
orders['order_month'] = orders['order_purchase_timestamp'].dt.to_period('M')
```

✓ [31] 28ms

```
# Merge orders with order_items
orders_items = orders.merge(order_items, on='order_id', how='left')
✓ [32] 85ms
```

```
# Group by month and sum sales
monthly_sales = orders_items.groupby('order_month')['price'].sum()
✓ [33] 28ms
```

```
# Line chart
monthly_sales.plot(kind='line', marker='o', title='Monthly Sales Trend')
plt.xlabel("Month")
plt.ylabel("Total Sales Value")
plt.show()
✓ [34] 152ms
```



```
# Highest and lowest months
print("Highest Sales Month:", monthly_sales.idxmax(), "with value:", monthly_sales.max())
print("Lowest Sales Month:", monthly_sales.idxmin(), "with value:", monthly_sales.min())
```

✓ [35] 49ms

```
Highest Sales Month: 2017-11 with value: 1010271.37
Lowest Sales Month: 2018-10 with value: 0.0
```

```
# Count customers by state
state_counts = customers['customer_state'].value_counts()
```

✓ [36] 34ms

```
# Top 10 states
top10_states = state_counts.head(10)
print(top10_states)
```

✓ [37] 37ms

```
customer_state
SP      41746
RJ      12852
MG      11635
RS       5466
PR       5045
SC       3637
BA       3380
DF       2140
ES       2033
GO       2020
Name: count, dtype: int64
```

```
# Percentage of customers from each state
state_percentage = (state_counts / state_counts.sum()) * 100
print(state_percentage.head(10))
```

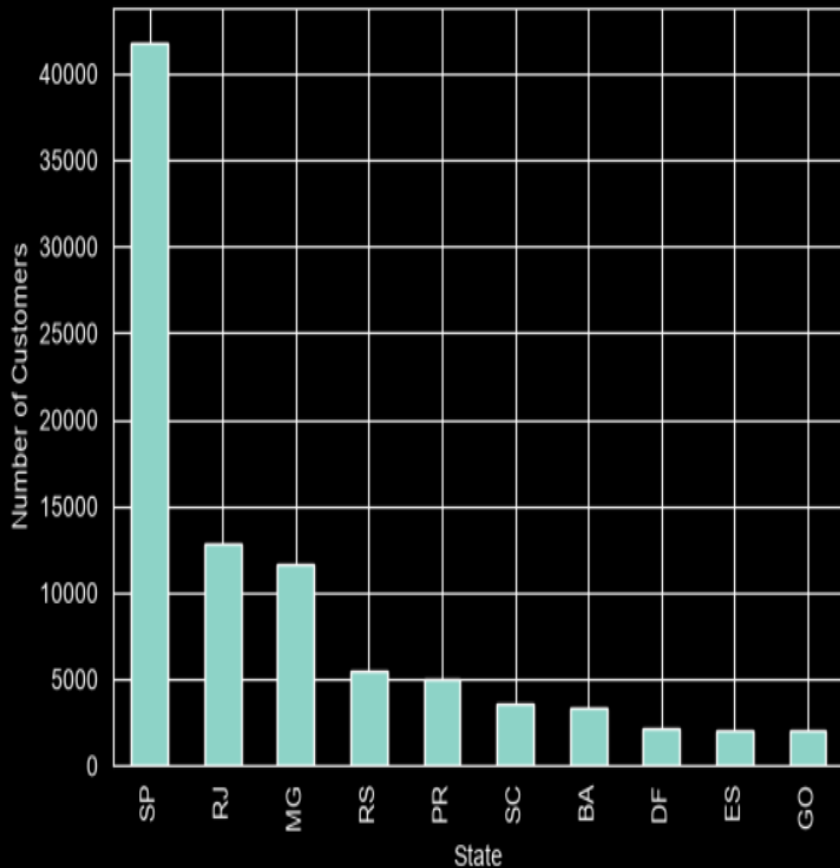
✓ [38] 37ms

```
customer_state
SP      41.980672
RJ      12.924247
MG      11.700405
RS       5.496727
PR       5.073360
SC       3.657445
BA       3.399000
DF       2.152030
ES       2.044428
GO       2.031355
Name: count, dtype: float64
```

```
# Bar chart
top10_states.plot(kind='bar', title='Top 10 States by Number of Customers')
plt.xlabel("State")
plt.ylabel("Number of Customers")
plt.show()
```

✓ [39] 143ms

Top 10 States by Number of Customers



```
# Merge customers with orders
cust_orders = customers.merge(orders, on='customer_id', how='left')
```

✓ [40] 73ms

```
# Merge with order_items
```

```
cust_orders_items = cust_orders.merge(order_items, on='order_id', how='left')
```

✓ [41] 94ms

```
# Calculate total spending per customer
```

```
customer_spending = cust_orders_items.groupby('customer_id')['price'].sum().sort_values(ascending=False)
```

✓ [42] 100ms

[Code](#) [Markdown](#)



```
# Top 10 customers
```

```
top10_spenders = customer_spending.head(10)
```

```
print(top10_spenders)
```

```
customer_orders = cust_orders_items.groupby('customer_id')['order_id'].nunique()
```

```
top_customers = pd.DataFrame({
    'Total Spending': customer_spending,
    'Number of Orders': customer_orders
}).sort_values(by='Total Spending', ascending=False).head(10)
```

```
print(top_customers)
```

✓ [43] 281ms

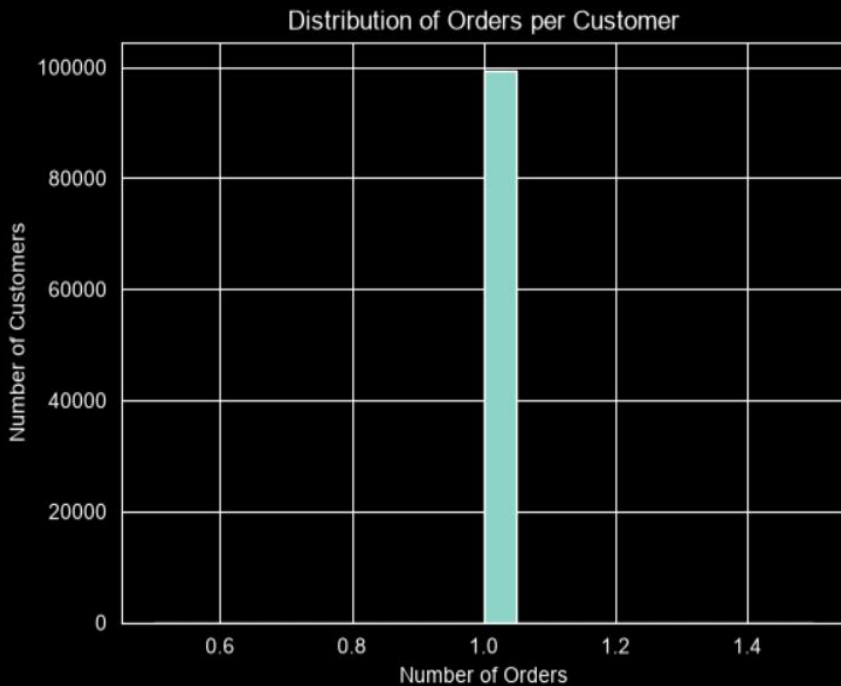
```
customer_id
1617b1357756262bfa56ab541c47bc16    13440.0
ec5b2ba62e574342386871631fafd3fc     7160.0
c6e2731c5b391845f6800c97401a43a9     6735.0
f48d464a0baaea338cb25f816991ab1f     6729.0
3fd6777bbce08a352fddd04e4a7cc8f6     6499.0
05455dfa7cd02f13d132aa7a6a9729c6     5934.6
df55c14d1476a9a3467f131269c2477f     4799.0
24bbf5fd2f2e1b359ee7de94defc4a15     4690.0
e0a2412720e9ea4f26c1ac985f6a7358     4599.9
3d979689f636322c62418b6346b1c6d2     4590.0
Name: price, dtype: float64
```

```
# Orders per customer
orders_per_customer = cust_orders.groupby('customer_id')['order_id'].nunique()
✓ [44] 114ms
```

```
# Average, max, min
print("Average Orders per Customer:", orders_per_customer.mean())
print("Max Orders by One Customer:", orders_per_customer.max())
print("Min Orders by One Customer:", orders_per_customer.min())
✓ [45] 39ms
```

```
Average Orders per Customer: 1.0
Max Orders by One Customer: 1
Min Orders by One Customer: 1
```

```
# Histogram
orders_per_customer.plot(kind='hist', bins=20, title='Distribution of Orders per Customer')
plt.xlabel("Number of Orders")
plt.ylabel("Number of Customers")
plt.show()
✓ [46] 148ms
```



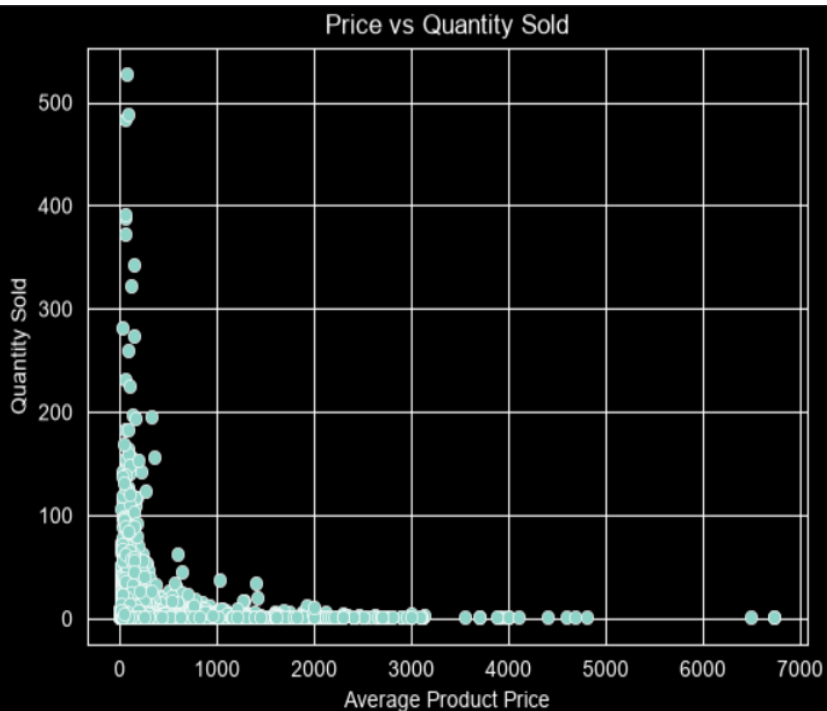
```
import seaborn as sns

# Group by product and calculate total quantity + average price
price_quantity = order_items.groupby('product_id').agg({
    'price': 'mean',
    'order_item_id': 'count'
}).reset_index()
✓ [48] 48ms

# Rename for clarity
price_quantity.rename(columns={'order_item_id': 'quantity'}, inplace=True)

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
✓ [50] 10ms

# Scatter plot
sns.scatterplot(x='price', y='quantity', data=price_quantity)
plt.title("Price vs Quantity Sold")
plt.xlabel("Average Product Price")
plt.ylabel("Quantity Sold")
plt.show()
✓ [51] 244ms
```



⚠ 9 ✓ 10

```
# Correlation
correlation = price_quantity['price'].corr(price_quantity['quantity'])
print("Correlation between Price and Quantity:", correlation)
✓ [52] 45ms

Correlation between Price and Quantity: -0.03213986268094516
```

```
# Convert date columns to datetime
orders['order_purchase_timestamp'] = pd.to_datetime(
    orders['order_purchase_timestamp'],
    errors='coerce'
)

orders['order_delivered_customer_date'] = pd.to_datetime(
    orders['order_delivered_customer_date'],
    errors='coerce'
)

orders['order_estimated_delivery_date'] = pd.to_datetime(
    orders['order_estimated_delivery_date'],
    errors='coerce'
)
```

✓ [53] 95ms

```
# Calculate delivery time in days
orders['delivery_time'] = (
    orders['order_delivered_customer_date']
    - orders['order_purchase_timestamp']
).dt.days
```

✓ [54] 27ms

```
# Summary statistics
print("Average Delivery Time:", orders['delivery_time'].mean())
print("Minimum Delivery Time:", orders['delivery_time'].min())
print("Maximum Delivery Time:", orders['delivery_time'].max())
print("Median Delivery Time:", orders['delivery_time'].median())
```

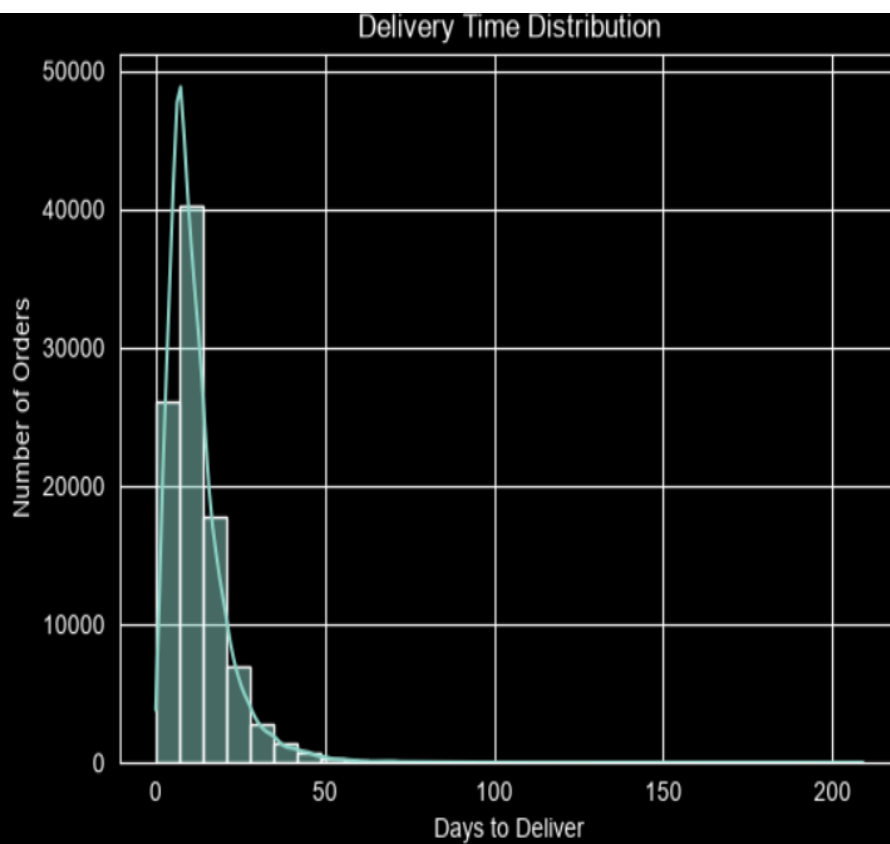
✓ [55] 34ms

```
Average Delivery Time: 12.094085575687217
Minimum Delivery Time: 0.0
Maximum Delivery Time: 209.0
Median Delivery Time: 10.0
```

```
# Distribution chart
sns.histplot(
    orders['delivery_time'].dropna(),
    bins=30,
    kde=True
)

plt.title("Delivery Time Distribution")
plt.xlabel("Days to Deliver")
plt.ylabel("Number of Orders")
plt.show()
```

✓ [56] 430ms



```
# Compare actual vs estimated delivery
orders['late_delivery'] = (
    orders['order_delivered_customer_date']
    > orders['order_estimated_delivery_date']
)

late_count = orders['late_delivery'].sum()

print("Number of Late Deliveries:", late_count)
```

✓ [57] 42ms

Number of Late Deliveries: 7827

```
# Create final cleaned dataset
cleaned_data = orders.copy()
```

✓ [58] 37ms

```
# Save cleaned dataset
cleaned_data.to_csv("Swati_Project1_Cleaned_Data.csv", index=False)

print("Cleaned data saved successfully.")
```

✓ [59] 427ms

Cleaned data saved successfully.

```
# Example: exporting cleaned orders dataset
orders.to_csv("Swati_Project1_Cleaned_Data.csv", index=False)
```

✓ [60] 442ms